

EatEasy EE — Architektur

Modularer Monolith · Vue 3 + Quarkus · Studienprojekt Gruppe 5

Grundsatzentscheidungen

■ Architekturstil: Modularer Monolith

- Fachliche Komponenten statt technischer Layer
- Kommunikation nur ueber Service-Interfaces (CDI), nie via REST

■ Strikte DTO-Trennung

- Entities lecken nie aus REST-Endpunkten; DTOs sind Java-Records

■ Auth: Stateless JWT (8 h)

- @RolesAllowed plus Authorization-Checks im Service-Layer

■ Daten: UUID v4, Flyway-Migrationen, Hibernate = validate

Komponenten & Abhaengigkeiten

■ Backend (de.eateasy.*)

- auth · household · recipe · ingredient · mealplan
- pantry · shoppinglist · suggestion · integration · notification · common

■ Schichten je Komponente

- entity -> repository -> service (Interface+Impl) -> resource -> dto

■ Abhaengigkeitsfluss

- auth -> household -> (recipe, pantry) -> mealplan -> shoppinglist -> suggestion
- notification an household; integration speist recipe & pantry

Genutzte Technologien

- Frontend: Vue 3, Vite, Pinia, Vue Router, TypeScript, Tailwind, VueUse
- Backend: Quarkus (Java 21), RESTEasy Reactive, Hibernate Panache, SmallRye JWT
- Datenbank: PostgreSQL 16 + Flyway
- LLM: Ollama self-hosted (Llama 3 / Mistral)
- Externe APIs: TheMealDB (Rezepte), OpenFoodFacts (Barcode)
- Testing: JUnit5 + REST Assured, Vitest + MSW, Playwright (E2E)
- Qualitaet/CI: Checkstyle, ESLint + Prettier, GitHub Actions
- Deployment: Render (Backend + DB), Vercel (Frontend, geplant)

Resilienz / Backup-Plan

- Smart-Suggestion: Ollama-Ausfall -> Coverage-Heuristik
- Barcode-Scan: keine Kamera -> manuelle Eingabe (optional)
- Rezept-Import: nur TheMealDB (kostenlos, kein API-Key)
- E-Mail: Invitation wird auch bei Mail-Fehler angelegt

Was wir noch planen (Issues #3-#8)

- **#3 Auto-Haushalt bei Registrierung**
 - AuthService injiziert HouseholdService, eine @Transactional-Einheit
- **#4 Rezept-Rating (1-5 Sterne) — einziger Schema-Eingriff (V8)**
- **#5 Einkaufsliste explizit gegen Vorrat prüfen vor Ausstellen**
- **#6 LLM-Modell verifizieren / leichteres Modell**
- **#7 Vercel-Deployment fürs Frontend**
- **#8 Backup-Plan, Abhängigkeiten & Scope dokumentieren**

Kernaussage

- **Architektur bleibt unverändert stabil:**

- Modularer Monolith, Service-Interfaces, strikte DTO-Trennung
- JWT + Service-Layer-Authz, Flyway, UUID

- **Nur #4 (Rating) beruehrt das Datenmodell**

- **Alles andere ist Service-Logik, Config oder Deployment/Doku**